TorchDCM: A PyTorch-First Package for Discrete Choice Model Estimation, Inference, and Benchmarking

Baichuan Mo

July 5, 2026

Abstract

Discrete choice models are central to transportation, marketing, operations, and service-system research, but researchers who need both econometric inference and modern tensor computation still face a software gap. Mature packages such as Biogeme, Apollo, and `mlogit` provide rich model syntax and inference, whereas tensor frameworks provide vectorized automatic differentiation but do not expose a discrete-choice-oriented estimation and benchmarking workflow. We introduce TorchDCM, a PyTorch-first package for discrete choice model estimation, inference, post-estimation analysis, and reproducible estimator benchmarking. TorchDCM implements a model zoo that includes multinomial logit, nested logit, cross-nested logit, mixed logit, WTP-space mixed logit, ordered response models, latent class models, and hybrid choice components, while retaining familiar econometric outputs such as covariance matrices, standard errors, willingness-to-pay measures, elasticities, and probability predictions. The package is paired with a public benchmark system that aligns data, model specifications, starting values, covariance definitions, probability calculations, and runtime accounting against Biogeme, Apollo, SciPy, `mlogit`, `gmnl`, and `xlogit`. On current public benchmarks, TorchDCM matches reference estimators to numerical tolerance for full-estimation MNL, nested logit, cross-nested logit, ordered logit, and ordered probit cases, and it verifies mixed-logit likelihood kernels using shared simulation draws. Across the full Swissmetro public-data benchmark after common filtering, TorchDCM completes full-estimation MNL in 0.030 seconds versus 1.188 seconds for the fastest econometric reference, nested logit in 0.069 seconds versus 2.100 seconds, and cross-nested logit in 1.108 seconds versus 3.870 seconds. On an NHTS 2022 public travel-survey mode-choice benchmark, TorchDCM estimates a 27,375-trip, 16-parameter MNL in 0.121 seconds versus 1.735 seconds for Biogeme. These results position TorchDCM as open infrastructure for choice-model research that needs differentiable computation, econometric reporting, and transparent estimator parity checks.

1 Introduction

Discrete choice models provide a statistical language for understanding decisions among finite alternatives. They are routinely used to study travel mode choice, route choice, product demand, pricing, and service operations, where researchers need both interpretable utility parameters and reliable counterfactual predictions [5, 8]. As empirical choice data become larger and model structures become more heterogeneous, the computational requirements of choice modeling increasingly resemble those of modern differentiable systems: likelihoods must be vectorized, simulation draws must be reproducible, and gradients, Hessians, predictions, and post-estimation quantities must remain aligned.

Existing software serves important parts of this workflow, but no single open package currently combines PyTorch-native computation with econometric model abstractions and a systematic estimator benchmark suite. Biogeme provides a mature Python environment for discrete choice model specification and estimation [2]; Apollo provides a flexible R framework for a broad range of choice models [4]; and R packages such as `mlogit` and `gmnl` provide widely used estimators for random utility models [3, 7]. These packages remain essential reference implementations, yet their computational backends and interfaces are not designed around PyTorch tensors, automatic differentiation pipelines, or direct integration with modern machine-learning workflows. Conversely, PyTorch provides efficient tensor computation and automatic differentiation [6], but it does not itself provide the data structures, model syntax, inference outputs, and benchmark discipline expected in discrete choice research.

TorchDCM addresses this gap by treating software design and estimator validation as a single contribution. The package keeps the model implementation in a pure Python/PyTorch library while placing heavyweight validation, reference-estimator wrappers, and public benchmark reports in a separate validation layer. This separation is deliberate: the user-facing package should remain lightweight and installable, while the benchmark system can depend on Biogeme, Apollo, R packages, and large public datasets. The result is a package that is both usable as a modeling library and auditable as an estimator implementation.

The design of TorchDCM follows three principles. First, the package uses discrete-choice-native abstractions, including choice datasets, utility specifications, beta parameters, nests, random coefficients, thresholds, latent classes, and post-estimation results. Second, the likelihood kernels are written as vectorized PyTorch computations, allowing a common implementation strategy for ragged choice sets, availability masks, panel likelihoods, simulation draws, and Hessian-based inference. Third, every implemented estimator is developed together with benchmark cases that compare log likelihoods, parameters, probabilities, covariance matrices, standard errors, willingness-to-pay measures, elasticities, and runtime splits against external reference software.

This paper makes three contributions. First, we introduce TorchDCM as an open-source PyTorch-first package for discrete choice estimation and inference. Second, we document the package architecture and implementation choices needed to support both standard and advanced choice models, including multinomial, nested, cross-nested, mixed, WTP-space, ordered, latent class, and hybrid-choice components. Third, we build a reproducible benchmark suite on public datasets aligned with Biogeme, Apollo, and R package examples, and we report estimator parity and runtime results across model families. The current benchmark suite establishes full-estimation parity for MNL, nested logit, cross-nested logit, ordered logit, and ordered probit models, and it establishes shared-draw likelihood parity for mixed-logit models.

The rest of the paper is organized as follows. Section 2 positions TorchDCM relative to existing choice-modeling software. Section 3 describes the package design. Section 4 presents implementation details for data handling, likelihood computation, inference, and simulation. Section 5 describes the benchmark data and alignment protocol. Section 6 reports computational results. Sections 7 and 8 discuss current limitations and future work.

2 Related Software

Discrete choice software has a long tradition of emphasizing model specification, estimation, and econometric reporting. Biogeme is a Python-based reference platform for discrete choice models and is widely used for transportation and choice-modeling research [2]. Apollo is a flexible R package that supports a broad set of discrete choice models, including advanced structures such as mixed logit, latent class, and hybrid choice models [4]. TorchDCM does not replace these tools; instead, it uses them as reference implementations in a benchmark system and focuses on PyTorch-native computation.

R packages provide another important software line for random utility modeling. The `mlogit` package provides an established interface for multinomial logit and related random utility models in R [3]. The `gmnl` package extends this ecosystem toward generalized multinomial logit and heterogeneous preference models [7]. These packages are useful baselines because they are widely cited, stable, and attached to public example datasets. TorchDCM therefore includes validation wrappers that compare parameters, covariance matrices, standard errors, and t-statistics against these packages on their canonical data examples.

Python choice-modeling packages increasingly target performance and machine-learning integration. The `xlogit` package provides GPU-accelerated estimation for mixed logit models and is an important Python reference for simulation-based choice modeling [1]. Other packages such as PyLogit and ML-oriented choice libraries broaden the Python ecosystem, but they do not provide the same combination of PyTorch-native tensors, broad econometric inference outputs, and cross-package benchmark reports targeted by TorchDCM. The closest conceptual relationship is therefore not a single package, but the combination of econometric model interfaces, tensor differentiation, and reproducible software benchmarking.

Software papers in operations research often contribute by turning complex algorithms into open, modular, and benchmarked tools. Recent IJOC-style software papers follow this pattern by identifying an implementation gap, documenting package design choices, and reporting reproducible computational comparisons. TorchDCM follows the same software contribution logic: the paper is not only about a new estimator, but about an extensible implementation and a benchmark system that allows researchers to verify estimator behavior across packages.

3 Package Design

TorchDCM is organized around a lightweight package layer and a separate validation layer. The package layer contains installable code under `torchdcm/`, including data containers, specification objects, model classes, inference utilities, and result objects. The validation layer contains public data processing, reference-estimator wrappers, benchmark scripts, and generated reports. This organization keeps the user-facing package compact while preserving the ability to run heavyweight comparisons against Biogeme, Apollo, and R packages.

The central user-facing abstraction is a choice dataset. For standard choice models, a `ChoiceDataset` stores alternatives, chosen alternatives, variables, availability masks, optional weights, and optional individual identifiers for panel likelihoods. The package supports wide-to-long conversion for common transportation data layouts and ragged long-format choice sets when different observations have different feasible alternatives. For ordered response models, an `OrderedChoiceDataset` stores ordinal outcomes, covariates, category labels, and optional observation weights.

Model specification is expressed through utility objects and named parameters. Users define `Beta` parameters and combine them with observed variables in a `UtilitySpec`. This syntax keeps the model readable while preserving a tensor-executable representation of each utility expression. Advanced model objects extend the same specification layer: nested logit adds nests and dissimilarity parameters; cross-nested logit adds allocation weights; mixed logit adds random coefficients and simulation draws; ordered models add thresholds; and latent class models add class-specific utilities and membership models.

Result objects are designed to expose econometric outputs rather than only fitted tensors. A fitted model returns parameter estimates, log likelihoods, covariance estimates, standard errors, t-statistics, probability predictions, and post-estimation summaries when available. For MNL and related models, the package also reports willingness-to-pay and elasticity calculations used in the benchmark suite. This reporting design is important because the package is evaluated not only by final likelihood values but also by whether downstream econometric quantities match reference implementations.

4 Implementation

TorchDCM implements likelihood evaluation as vectorized PyTorch tensor computation. Each model maps a data object and parameter vector to utilities, probabilities, and log-likelihood contributions. Availability masks are applied before normalization, so unavailable alternatives receive zero probability while the feasible set is normalized correctly. For panel data, row-level probabilities are grouped by individual identifiers and multiplied through the log domain to form individual-level likelihood contributions.

The package uses automatic differentiation for gradients and Hessian-based inference. Parameter objects define initial values, fixed/free status, and transformations when constraints are needed. For unconstrained models, optimization proceeds over free parameters directly. For constrained parameters such as nest dissimilarities, the implementation optimizes an unconstrained internal representation and maps it to the feasible interval during likelihood evaluation. After estimation, the package computes covariance matrices from observed information where available and exposes standard errors and t-statistics through the result object.

Simulation-based models use explicit draw tensors. Mixed logit and WTP-space mixed logit average probabilities over deterministic or seeded simulation draws, and panel likelihoods share draws consistently across observations belonging to the same individual. This design is essential for cross-package validation: when TorchDCM, Biogeme, and Apollo receive the same parameters and the same draw matrix, probability and likelihood differences should isolate implementation errors rather than random simulation noise. The current benchmark suite uses this design to verify mixed-logit likelihood kernels before extending the comparison to full simulated maximum-likelihood estimation.

The validation layer is implemented as executable benchmark scripts rather than static tables. Each benchmark case records data source, model specification, starting values, availability rules, scaling choices, covariance definition, backend availability, runtime split, and numerical differences. Generated JSON and Markdown reports are treated as experiment artifacts. This mirrors the software-paper pattern in which the package is evaluated as a reproducible computational system, not only as a collection of model classes.

5 Benchmark Data and Protocol

The benchmark protocol follows the principle that estimator comparisons are meaningful only when data, specifications, starting values, and post-estimation definitions are aligned. For each case, the validation layer records the public data source, alternatives, variables, availability filters, scaling transformations, initial parameter values, covariance definition, and reference backend. When a backend exposes separate timing, the protocol reports parameter-estimation time and covariance/Hessian time separately.

The benchmark suite currently uses three groups of public data. The first group contains public Biogeme datasets, including airline itinerary choice, parking choice, telephone service choice, Swissmetro, Optima, and London Passenger Mode Choice. The second group contains R community datasets used by `mlogit`, including Fishing and ModeCanada. The third group contains processed large-data candidates that are maintained as choice-set artifacts rather than raw survey dumps. Small public datasets are committed to the repository, while large processed artifacts are distributed separately.

The numerical metrics are chosen to match the Apollo benchmark plan used to guide the project, but the paper tables report a single consistency flag rather than every raw difference. A full-estimation case is marked consistent when all directly comparable quantities satisfy the following prespecified tolerances: absolute log-likelihood difference at most 10^{-4}, maximum absolute parameter difference at most 5×10^{-4}, maximum probability difference at most 2×10^{-4}, and maximum standard-error difference at most 3×10^{-2} when the same covariance definition is available. Willingness-to-pay and elasticity checks use the same relative numerical scale when the model specification supports them. For simulation-based models that have not yet been fully estimated across all backends, the suite uses fixed-parameter replay with shared draws and marks consistency for the likelihood/probability kernel rather than for the complete simulated estimator.

The runtime metrics distinguish estimator work from reporting work. Total runtime includes backend overhead when the benchmark script can measure it, while estimation time isolates parameter optimization and covariance time isolates Hessian or covariance calculations. This split is important because some reference packages perform model construction, reporting, file output, or post-processing outside the core optimizer. The paper-facing benchmark tables therefore report total runtime, while the finer estimation/covariance split remains available in the generated validation artifacts.

6 Computational Results

The computational results use two complementary benchmark families. The first family uses public empirical datasets aligned with Biogeme, Apollo, and R package examples. The second family is a pure synthetic controlled benchmark that varies the number of samples, alternatives, variables, and feature correlations under a known MNL data-generating process. The empirical results are organized by model family: MNL in Table 1, nested-logit-family models in Table 2, and mixed-logit models in Table 3.

Table 1: Real-data MNL runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for each aligned estimator. A dash means that no aligned wrapper is included for that dataset-estimator pair; *Fail* means the wrapper was attempted but did not complete.

Data	N	TorchDCM	SciPy	Biogeme	Apollo	`mlogit`	`gmnl`	`xlogit`	Consistent?
Swissmetro	10719	0.028	2.328	1.187	1.825	0.692	*Fail*	0.029	Yes
NHTS 2022	27375	0.099	44.345	1.752	9.843	2.898	31.955	0.711	Yes
Airline itinerary	3609	0.025	3.103	1.195	1.199	0.585	0.787	0.011	Yes
Parking Spain	1576	0.023	1.873	1.174	1.129	0.557	0.698	0.006	Yes
Telephone service	434	0.022	0.241	1.181	1.094	0.544	*Fail*	0.004	Yes
LPMC London	81086	0.074	59.196	1.398	21.171	4.000	7.769	0.325	Yes
Catsup	2798	0.021	0.375	1.158	1.120	0.053	0.705	0.008	Yes
Cracker	3292	0.028	0.544	1.185	1.138	0.057	0.716	0.009	Yes
Electricity	4308	0.032	2.821	1.592	1.282	0.197	0.970	0.013	Yes
HC	250	0.020	0.048	1.365	1.045	0.017	0.667	0.002	Yes
Heating	900	0.019	0.140	1.016	1.055	0.025	0.667	0.003	Yes
Mode	453	0.017	0.201	0.952	1.026	0.016	0.663	0.002	Yes
NOx	632	0.020	0.238	2.426	1.132	0.041	*Fail*	0.004	Yes
RiskyTransport	1793	0.038	1.147	1.815	1.214	0.045	*Fail*	*Fail*	Yes
Train	2929	0.032	1.771	0.886	1.137	0.033	0.705	0.005	Yes
Fishing	1182	0.033	0.813	1.176	1.101	1.176	0.707	0.006	Yes

Continued on next page

Table 1: Real-data MNL runtime and consistency benchmarks, continued.

Data	N	TorchDCM	SciPy	Biogeme	Apollo	`mlogit`	`gmnl`	`xlogit`	Consistent?
ModeCanada	4324	0.028	1.960	1.155	1.200	0.613	*Fail*	*Fail*	Yes

Table 1 reports the main real-data MNL benchmark. Each row uses an actual public dataset and an aligned model specification. The columns report total remote wall-clock runtime for TorchDCM and every available benchmark estimator; the final column reports whether all successful aligned external estimators agree with TorchDCM under the prespecified tolerance rule. The table includes Swissmetro, NHTS 2022, four Biogeme public MNL datasets, and eleven R `mlogit` package datasets. Biogeme and Apollo are now run for every MNL row, while `mlogit`, `gmnl`, and `xlogit` are included where aligned package wrappers are available. `xlogit` is especially fast on small standard MNL rows because the wrapper passes a prebuilt long-format numeric design matrix directly to a specialized in-process MNL solver; the comparison therefore separates this low-overhead standard-MNL path from broader package coverage such as symbolic specification, ragged choice sets, and non-MNL models. ModeCanada retains attempted failures for `gmnl` and `xlogit`; these failures are not treated as consistency failures for the successful aligned estimators.

Table 2: Real-data nested-logit-family runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for each aligned estimator.

Data	Model	N	TorchDCM	Biogeme	Apollo	Consistent?
Swissmetro	Nested logit	10719	0.078	2.968	2.083	Yes
Swissmetro	Cross-nested logit	10719	1.106	3.839	–	Yes

Table 2 reports the current real-data nested-logit-family benchmark. The Swissmetro nested-logit row is compared with both Biogeme and Apollo. The cross-nested row is currently compared with Biogeme under fixed allocation weights and common nest constraints. Both rows are consistent under the prespecified tolerance rule.

Table 3: Real-data mixed-logit runtime and consistency benchmarks. Current rows are shared-parameter, shared-draw likelihood/probability replay benchmarks on Swissmetro; full simulated maximum-likelihood estimation remains separate from this table. Runtime columns report total remote wall-clock seconds.

Data	Model	N	TorchDCM	Biogeme	Apollo	Consistent?
Swissmetro	Mixed logit replay	10719	0.016	13.073	0.585	Yes
Swissmetro	WTP-space mixed logit replay	10719	0.017	13.644	0.590	Yes

Table 3 reports the current real-data mixed-logit benchmark. These rows use Swissmetro with shared parameters and shared antithetic draws, so they validate the mixed-logit likelihood and probability kernels rather than full simulated maximum-likelihood estimation. TorchDCM matches both Biogeme and Apollo to numerical tolerance in the available shared-draw replay rows.

Table 4: Benchmark case scale and alignment mode. The runtime table reports only estimator wall-clock time and consistency; this table records sample size, parameter count, and whether the row is full estimation or a shared-parameter replay.

Case	Model	N	K	Alignment mode
Swissmetro	MNL	10719	4	Full estimation with shared zero initial values.
Swissmetro	Nested logit	10719	5	Full estimation with shared zero initial values and common nest constraints.
Swissmetro	Cross-nested logit	10719	6	Full estimation with fixed allocation weights and common nest constraints.
Swissmetro	Mixed logit replay	10719	5	Fixed-parameter likelihood/probability replay with 64 shared antithetic draws and panel likelihood.
Swissmetro	WTP mixed replay	10719	5	Fixed-parameter WTP-space likelihood/probability replay with 64 shared antithetic draws and panel likelihood.
Optima	Ordered logit	1822	9	Full estimation on the aligned Biogeme Optima ordered-response specification.
Optima	Ordered probit	1822	9	Full estimation on the aligned Biogeme Optima ordered-response specification.
Airline itinerary	MNL	3609	5	Full estimation on the Biogeme airline itinerary choice data.
Parking Spain	MNL	1576	5	Full estimation on the Biogeme parking choice data.
Telephone service	MNL	434	5	Full estimation on the Biogeme telephone-service choice data.
LPMC London	MNL	81086	5	Full estimation on the London Passenger Mode Choice data.
NHTS 2022	MNL	27375	16	Full estimation on a processed public-use trip-mode choice benchmark.
Fishing	MNL	1182	5	Full estimation against R `mlogit`, `gmnl`, and `xlogit`.
ModeCanada	MNL	4324	3	Full estimation against R `mlogit`.
Catsup	MNL	2798	3	Full estimation against R `mlogit`.
Cracker	MNL	3292	3	Full estimation against R `mlogit`.
Electricity	MNL	4308	6	Full estimation against R `mlogit`.
HC	MNL	250	2	Full estimation against R `mlogit`.
Heating	MNL	900	2	Full estimation against R `mlogit`.
Mode	MNL	453	2	Full estimation against R `mlogit`.
NOx	MNL	632	3	Full estimation against R `mlogit`.
RiskyTransport	MNL	1793	7	Full estimation against R `mlogit`.
Train	MNL	2929	4	Full estimation against R `mlogit`.

Table 4 records the corresponding parameter counts and alignment modes. The raw numerical-difference audit, including likelihood, parameter, probability, covariance, and standard-error differences, is retained in the generated validation artifacts rather than repeated in the paper-facing runtime tables.

Table 5: Pure synthetic controlled MNL runtime benchmarks. These cases are generated from a synthetic data-generating process and are not based on Swissmetro or any public empirical dataset. The benchmark varies sample size N, number of alternatives J, number of variables K, and feature correlation ρ, and reports total TorchDCM runtime.

Sweep	Case	N	J	K	ρ	Params	Total (s)	Consistent?
Sample size	$N = 1{,}000$	1000	4	6	0.30	9	0.021	Yes
Sample size	$N = 10{,}000$	10000	4	6	0.30	9	0.012	Yes
Sample size	$N = 100{,}000$	100000	4	6	0.30	9	0.068	Yes
Alternatives	$J = 3$	20000	3	6	0.30	8	0.014	Yes
Alternatives	$J = 20$	20000	20	6	0.30	25	0.409	Yes
Variables	$K = 4$	20000	5	4	0.30	8	0.021	Yes
Variables	$K = 32$	20000	5	32	0.30	36	0.113	Yes
Correlation	$\rho = 0.00$	20000	5	12	0.00	16	0.031	Yes
Correlation	$\rho = 0.98$	20000	5	12	0.98	16	0.054	Yes

Table 5 shows the pure synthetic scaling results using total runtime only. The synthetic cases are not derived from Swissmetro or any other public empirical dataset; covariates, utility parameters, and choices are generated directly from a synthetic MNL data-generating process. The sample-size sweep shows that TorchDCM remains fast as N grows: the $N = 100{,}000$ case has 400,000 long rows and completes in 0.068 seconds including covariance calculation. The variable-count sweep shows the effect of increasing the number of observed variables: the $K = 32$ case estimates 36 total parameters and completes in 0.113 seconds including covariance. The alternative-count sweep is also included because choice-set width is a central computational dimension for DCM estimators: the $J = 20$ case has 400,000 long rows and completes in 0.409 seconds including covariance. These synthetic experiments complement the public-data estimator comparisons by isolating controlled scaling behavior.

Ordered response models are validated on the Biogeme Optima data. TorchDCM completes ordered logit full estimation in 0.042 seconds compared with 2.898 seconds for Biogeme. For ordered probit, TorchDCM completes estimation in 0.054 seconds compared with 3.475 seconds for Biogeme. Both ordered-response rows are marked consistent with the Biogeme reference implementation.

7 Discussion and Limitations

The current results support the main software claim that TorchDCM can combine PyTorch-native likelihood computation with econometric outputs and external estimator parity checks. The strongest evidence comes from full-estimation MNL and ordered-model cases, where the benchmark suite reports runtime splits and a prespecified consistency flag against reference estimators. Nested and cross-nested logit results extend this evidence to more structured substitution patterns, while mixed-logit fixed replay verifies the simulation likelihood kernels under shared draws.

The main limitation is that full mixed-logit estimation has not yet been completed across Biogeme, Apollo, and other mixed-logit packages. This limitation is a scope boundary rather than a contradiction of the current evidence: the present mixed-logit results validate the probability and log-likelihood kernels, but they do not validate optimizer convergence, simulated covariance, or random-draw conventions across packages. The next experimental step is therefore to estimate mixed-logit models end to end with deterministic shared draws and compare runtime plus consistency under the same rule.

A second limitation is that covariance comparisons for constrained models require careful interpretation. For nested and cross-nested logit, different packages may report covariance on transformed or constrained parameter scales, and numerical differences can be larger than likelihood or probability differences. The manuscript should therefore avoid claiming universal covariance equivalence until the transformation convention is audited and documented. The raw covariance and standard-error audits remain useful internally, but the paper-facing table should report only the final consistency flag.

The benchmark system is also still expanding in dataset coverage and controlled synthetic coverage. The current public data suite includes several Biogeme and R package examples, plus processed large-data candidates, and the current synthetic suite covers pure MNL runtime scaling under controlled dimensions. More Apollo examples, additional public large-scale choice datasets, and synthetic known-truth nested/mixed-logit cases should be added before final submission. This expansion will make the empirical section closer to IJOC software-paper standards, where reproducibility, public data, and baseline breadth are central acceptance signals.

8 Conclusion

This paper introduces TorchDCM, a PyTorch-first package for discrete choice model estimation, inference, and estimator benchmarking. The key idea is to pair tensor-native likelihood computation with econometric abstractions and a validation layer that compares against mature reference packages. Current public benchmarks show full-estimation numerical parity for MNL, nested logit, cross-nested logit, ordered logit, and ordered probit, and shared-draw likelihood parity for mixed-logit models. The strongest immediate evidence is that TorchDCM matches reference estimators on likelihoods, parameters, probabilities, covariance outputs, and standard errors while often reducing core runtime by one to two orders of magnitude on the tested cases.

Future work will focus on completing full mixed-logit estimation benchmarks, broadening Apollo and R package comparisons, and expanding the public dataset suite. These extensions will strengthen the package as open infrastructure for researchers who need differentiable choice models, econometric inference, and reproducible cross-estimator validation.

Reproducibility Statement

The TorchDCM repository separates installable package code from heavyweight validation. Package code is stored under `torchdcm/`, public small datasets under `datasets/small/`, documentation under `docs/`, and benchmark wrappers and generated results under `validation/`. The paper-facing solver matrix can be run on the remote benchmark machine with:

```
cd /home/baichuan-mo/torchdcm
.venv/bin/python validation/benchmarks/run_solver_attempt_matrix.py \
  --profile full \
  --python /home/baichuan-mo/torchdcm/.venv/bin/python
```

The paper-facing real-data benchmark tables are constructed from the remote solver-attempt matrix in `validation/generated/solver_attempt_matrix_full.json`, with detailed case-level audits retained in the generated validation files. Table-specific case scale and alignment details are recorded in `paper/tables/benchmark_cases.tex`. The broader Markdown benchmark summary is maintained in `docs/model-family-benchmark-comparison.md`.

References

[1] Camilo Arteaga, Jinwoo Park, and Alexander Paz. xlogit: An open-source Python package for GPU-accelerated estimation of mixed logit models. *Journal of Choice Modelling*, 42:100339, 2022.

[2] Michel Bierlaire. A short introduction to biogeme. Technical Report TRANSP-OR 230620, Transport and Mobility Laboratory, EPFL, 2023. URL https://biogeme.epfl.ch/.

[3] Yves Croissant. Estimation of random utility models in R: The mlogit package. *Journal of Statistical Software*, 95(11):1–41, 2020.

[4] Stephane Hess and David Palma. Apollo: A flexible, powerful and customisable freeware package for choice model estimation and application. *Journal of Choice Modelling*, 32:100170, 2019.

[5] Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974.

[6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[7] Mauricio Sarrias and Ricardo A. Daziano. Multinomial logit models with continuous and discrete individual heterogeneity in R: The gmnl package. *Journal of Statistical Software*, 79(2): 1–46, 2017.

[8] Kenneth E. Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, 2 edition, 2009.

A Claim-Evidence Map

Claim	Evidence in Current Draft	Status
TorchDCM is PyTorch-first and exposes discrete-choice abstractions.	Package design section describes `ChoiceDataset`, `UtilitySpec`, `Beta`, model classes, and result objects; repository structure supports this.	Supported
TorchDCM matches reference estimators for MNL full estimation.	Swissmetro, public Biogeme MNL battery, Fishing, and ModeCanada benchmark rows are marked consistent under the prespecified tolerance rule.	Supported
TorchDCM matches reference estimators for nested and cross-nested logit.	Swissmetro nested and cross-nested full-estimation rows are marked consistent under the prespecified tolerance rule.	Supported
TorchDCM validates mixed-logit kernels.	Shared-draw fixed replay against Biogeme and Apollo gives machine-precision probability and likelihood agreement.	Supported
TorchDCM has completed full mixed-logit estimator parity.	Current results only include fixed replay for mixed and WTP mixed logit.	Needs new experiment
TorchDCM provides faster runtime on current benchmark cases.	Tables report lower TorchDCM total or estimation time than Biogeme/Apollo/mlogit in most econometric-reference cases, with `xlogit` faster on Fishing MNL.	Supported with nuance
TorchDCM scales on pure synthetic MNL data.	Synthetic benchmark table reports runtime splits as N, J, K, and feature correlation vary.	Supported for MNL

B Reviewer-Facing Self-Review

Dimension	Reviewer-facing question	Current status
Contribution	Does the paper provide new software infrastructure rather than only a reimplementation?	Pass; clarify PyTorch-first plus benchmark-system contribution.
Writing clarity	Can a reader reproduce the package workflow and benchmark protocol?	Partial; add more API examples and exact environment details before submission.
Experimental strength	Are strong baselines included?	Partial; Biogeme, Apollo, SciPy, mlogit, gmnl, and xlogit are included, but mixed-logit full estimation is pending.
Evaluation completeness	Are all major model-family claims supported by full estimation?	Needs revision; mixed and WTP mixed currently use fixed replay.
Method soundness	Are hidden assumptions documented?	Partial; add constrained-parameter covariance convention and draw-generation details.